Business functions – syntax in SQL, Python and R

20 ArchiMate business functions. For each function: SQL (DuckDB), SQL (Trino), Python (pandas) and R (tidyverse).

Conventions

All examples use the same (fictitious) tables. Adapt names and thresholds to your own data.

Table / dataframe	Columns
`microdata`	`id`, `sbi`, `gemeente`, `leeftijd`, `beroep_tekst`, `omzet`, `kosten`, `winst`, `personeel`, `omzet_vorig`, `personeel_vorig`, `gewicht`
other tables	described with the function itself

Column and table names in the code are kept in Dutch, so the code matches the `.sql`, `.py` and `.R` files:

Name	Meaning
`sbi`	industry classification code (Dutch SIC)
`gemeente`	municipality
`leeftijd`	age
`beroep_tekst`	occupation (free text)
`omzet`	turnover
`kosten`	costs
`winst`	profit
`personeel`	employees
`*_vorig`	value in the previous period (T-1)
`gewicht`	(sampling) weight
`valide` / `plausibel`	valid / plausible
`geimputeerd`	imputed

- Python: `import pandas as pd, numpy as np`; the dataframe is called `df` (= `microdata`).
- R: `library(dplyr); library(tidyr); library(stringr); library(purrr); library(tibble)`; the data is called `microdata`.
- Where DuckDB and Trino share the same syntax there is a single "DuckDB & Trino" block.
- Differences that matter: integer division (Trino divides `bigint/bigint` as an integer, so always use `CAST(.. AS DOUBLE)`), regex (`regexp_matches` vs `regexp_like`), percentiles (`quantile_cont` vs `approx_percentile`), `median` (DuckDB only), `EXCLUDE`/`COLUMNS`/`UNION ALL BY NAME` (DuckDB only).
- In the checks below, a missing value yields `FALSE` (not valid / not plausible). Remove the `COALESCE` if you want to keep `NULL`.

---

1. Check validity

Checks a single variable against a fixed range; the result is an indicator (true/false). Repeat per variable.

SQL – DuckDB & Trino

```
SELECT m.*,
       COALESCE(leeftijd BETWEEN 0 AND 120, FALSE)       AS leeftijd_valide,
       COALESCE(sbi IN ('47111', '47112', '56101'), FALSE) AS sbi_valide      -- code list
FROM microdata m;
```

Python

```
df["leeftijd_valide"] = df["leeftijd"].between(0, 120)            # NaN -> False
df["sbi_valide"] = df["sbi"].isin(["47111", "47112", "56101"])
```

R

```
microdata |>
  mutate(leeftijd_valide = !is.na(leeftijd) & between(leeftijd, 0, 120),
         sbi_valide      = sbi %in% c("47111", "47112", "56101"))
```

---

2. Derive variables (typing)

Free text → category; only the value domain changes.

SQL – DuckDB

```
SELECT m.*,
       CASE WHEN beroep_tekst IS NULL THEN NULL
            WHEN regexp_matches(lower(beroep_tekst), 'verpleeg|arts|zorg')    THEN 'Zorg'
            WHEN regexp_matches(lower(beroep_tekst), 'leraar|docent|onderwijs') THEN 'Onderwijs'
            ELSE 'Overig' END AS beroepsgroep
FROM microdata m;
```

SQL – Trino

```
SELECT m.*,
       CASE WHEN beroep_tekst IS NULL THEN NULL
            WHEN regexp_like(lower(beroep_tekst), 'verpleeg|arts|zorg')    THEN 'Zorg'
            WHEN regexp_like(lower(beroep_tekst), 'leraar|docent|onderwijs') THEN 'Onderwijs'
            ELSE 'Overig' END AS beroepsgroep
FROM microdata m;
```

Python

```
t = df["beroep_tekst"].str.lower()
df["beroepsgroep"] = np.select(
    [t.str.contains("verpleeg|arts|zorg", na=False),
     t.str.contains("leraar|docent|onderwijs", na=False)],
    ["Zorg", "Onderwijs"], default="Overig")
df.loc[t.isna(), "beroepsgroep"] = None
```

R

```
microdata |>
  mutate(beroepsgroep = case_when(
    is.na(beroep_tekst)                                          ~ NA_character_,
    str_detect(str_to_lower(beroep_tekst), "verpleeg|arts|zorg")    ~ "Zorg",
    str_detect(str_to_lower(beroep_tekst), "leraar|docent|onderwijs") ~ "Onderwijs",
    TRUE                                                         ~ "Overig"))
```

> The optional "soft" outcome (probability distribution) requires a classifier (e.g. scikit-learn `predict_proba`, R `caret`/`tidymodels`); this cannot be done in SQL.

---

3. Derive variables (collapsing)

Detailed codes → collapsed codes via a code list `codelijst_indikking(code_detail, code_ingedikt)`.

SQL – DuckDB & Trino

```
SELECT m.*, k.code_ingedikt AS sbi_ingedikt
FROM microdata m
LEFT JOIN codelijst_indikking k ON m.sbi = k.code_detail;

-- variant for a hierarchical code: first 2 positions
SELECT m.*, substr(sbi, 1, 2) AS sbi_2 FROM microdata m;
```

Python

```
df = df.merge(codelijst_indikking.rename(columns={"code_detail": "sbi", "code_ingedikt": "sbi_ingedikt"}),
              on="sbi", how="left")
# variant: df["sbi_2"] = df["sbi"].str[:2]
```

R

```
microdata |>
  left_join(codelijst_indikking, by = c("sbi" = "code_detail")) |>
  rename(sbi_ingedikt = code_ingedikt)
# variant: mutate(sbi_2 = str_sub(sbi, 1, 2))
```

---

4. Check validity: general set of edit rules

Rules over combinations of variables, followed by error localization (Fellegi-Holt). Example rules: `omzet = kosten + winst`, `omzet >= 0`, `kosten >= 0`, `personeel >= 0`.

SQL – DuckDB & Trino (rule check + heuristic localization: for each violated rule, the variable that occurs in the most violated rules is flagged; exact Fellegi-Holt is not possible in SQL, see Python/R)

```
WITH regels AS (
  SELECT id, omzet, kosten, winst, personeel,
         COALESCE(ABS(omzet - (kosten + winst)) > 0.5, FALSE) AS r1,
         COALESCE(omzet     < 0, FALSE) AS r2,
         COALESCE(kosten    < 0, FALSE) AS r3,
         COALESCE(personeel < 0, FALSE) AS r4
  FROM microdata
), telling AS (
  SELECT *,
         CAST(r1 AS INTEGER) + CAST(r2 AS INTEGER) AS n_omzet,
         CAST(r1 AS INTEGER) + CAST(r3 AS INTEGER) AS n_kosten,
         CAST(r1 AS INTEGER)                       AS n_winst
  FROM regels
), fout AS (
  SELECT *,
         (r1 AND n_omzet  = GREATEST(n_omzet, n_kosten, n_winst)) OR r2 AS f_omzet,
         (r1 AND n_kosten = GREATEST(n_omzet, n_kosten, n_winst)) OR r3 AS f_kosten,
         (r1 AND n_winst  = GREATEST(n_omzet, n_kosten, n_winst))       AS f_winst,
         r4                                                             AS f_personeel
  FROM telling
)
SELECT id,
       NOT f_omzet     AS omzet_valide,
       NOT f_kosten    AS kosten_valide,
       NOT f_winst     AS winst_valide,
       NOT f_personeel AS personeel_valide
FROM fout;
```

Python (exact Fellegi-Holt for linear rules: the smallest set of variables that must be freed for all rules to hold; feasibility via LP)

```
import itertools
from scipy.optimize import linprog

variabelen = ["omzet", "kosten", "winst", "personeel"]
A_eq = np.array([[1, -1, -1, 0]]);  b_eq = np.array([0.0])               # omzet = kosten + winst
A_ub = np.array([[-1, 0, 0, 0], [0, -1, 0, 0], [0, 0, 0, -1]]); b_ub = np.zeros(3)  # omzet, kosten, personeel >=
0

def haalbaar(x, vrij):
    bounds = [(None, None) if i in vrij else (x[i], x[i]) for i in range(len(x))]
    res = linprog(np.zeros(len(x)), A_ub=A_ub, b_ub=b_ub, A_eq=A_eq, b_eq=b_eq,
                  bounds=bounds, method="highs")
    return res.status == 0

def lokaliseer(x):
    ontbrekend = {i for i in range(len(x)) if np.isnan(x[i])}      # missing = not an error, but free
    x0 = np.nan_to_num(x)
    if haalbaar(x0, ontbrekend):
        return set()
    waargenomen = [i for i in range(len(x)) if i not in ontbrekend]
    for k in range(1, len(waargenomen) + 1):                        # minimal cardinality
        for S in itertools.combinations(waargenomen, k):
            if haalbaar(x0, ontbrekend | set(S)):
                return set(S)                                       # first minimal set

fouten = [lokaliseer(x) for x in df[variabelen].to_numpy(float)]
for j, v in enumerate(variabelen):
    df[f"{v}_valide"] = [j not in f for f in fouten]
# optionally blank out: df.loc[~df["omzet_valide"], "omzet"] = np.nan
```

R (packages `validate` + `errorlocate`, Fellegi-Holt)

```
library(validate); library(errorlocate)

regels <- validator(omzet == kosten + winst, omzet >= 0, kosten >= 0, personeel >= 0)
dat    <- as.data.frame(microdata[, c("omzet", "kosten", "winst", "personeel")])

fout   <- locate_errors(dat, regels)$errors          # logical matrix, TRUE = erroneous value
valide <- as_tibble(!fout) |> rename_with(~ paste0(.x, "_valide"))
resultaat <- bind_cols(microdata, valide)
# optionally blank out: replace_errors(dat, regels)
```

---

5. Imputation

Ratio imputation: missing `omzet` = `personeel` × (Σomzet / Σpersoneel) within the SBI stratum of the responding units.

SQL – DuckDB & Trino

```
WITH verhouding AS (
  SELECT sbi, CAST(SUM(omzet) AS DOUBLE) / SUM(personeel) AS omzet_per_pers
  FROM microdata
  WHERE omzet IS NOT NULL AND personeel > 0
  GROUP BY sbi
)
SELECT m.*,
       COALESCE(m.omzet, m.personeel * v.omzet_per_pers)                          AS omzet_geimputeerd,
       (m.omzet IS NULL AND m.personeel * v.omzet_per_pers IS NOT NULL)           AS omzet_is_geimputeerd
FROM microdata m
LEFT JOIN verhouding v ON m.sbi = v.sbi;
```

Python

```
donor = df[df["omzet"].notna() & (df["personeel"] > 0)]
som = donor.groupby("sbi")[["omzet", "personeel"]].sum()
ratio = df["sbi"].map(som["omzet"] / som["personeel"])

df["omzet_is_geimputeerd"] = df["omzet"].isna() & (df["personeel"] * ratio).notna()
df["omzet_geimputeerd"] = df["omzet"].fillna(df["personeel"] * ratio)
```

R

```
ratio <- microdata |>
  filter(!is.na(omzet), personeel > 0) |>
  group_by(sbi) |>
  summarise(omzet_per_pers = sum(omzet) / sum(personeel), .groups = "drop")

microdata |>
  left_join(ratio, by = "sbi") |>
  mutate(omzet_is_geimputeerd = is.na(omzet) & !is.na(personeel * omzet_per_pers),
         omzet_geimputeerd    = coalesce(omzet, personeel * omzet_per_pers))
```

> Other methods (hot-deck, regression, PMM): Python `sklearn.impute` / `statsmodels`, R `VIM`, `mice`, `simputation`.

---

6. Derive variables (general)

New variable(s) from one or more existing ones; one example of each kind (deriving, changing measurement scale, collapsing, typing).

SQL – DuckDB

```
SELECT m.*,
       CAST(omzet AS DOUBLE) / NULLIF(personeel, 0)                                    AS omzet_per_werknemer,
       CASE WHEN personeel < 10 THEN 'klein' WHEN personeel < 100 THEN 'midden' ELSE 'groot' END AS
grootteklasse,
       substr(sbi, 1, 2)                                                              AS sbi_2,
       regexp_matches(lower(beroep_tekst), 'zorg|arts')                                AS is_zorg
FROM microdata m;
```

SQL – Trino

```
SELECT m.*,
       CAST(omzet AS DOUBLE) / NULLIF(personeel, 0)                                    AS omzet_per_werknemer,
       CASE WHEN personeel < 10 THEN 'klein' WHEN personeel < 100 THEN 'midden' ELSE 'groot' END AS
grootteklasse,
       substr(sbi, 1, 2)                                                              AS sbi_2,
       regexp_like(lower(beroep_tekst), 'zorg|arts')                                   AS is_zorg
FROM microdata m;
```

Python

```
df = df.assign(
    omzet_per_werknemer=df["omzet"] / df["personeel"].where(df["personeel"] != 0),
    grootteklasse=pd.cut(df["personeel"], [-np.inf, 10, 100, np.inf], labels=["klein", "midden", "groot"],
right=False),
    sbi_2=df["sbi"].str[:2],
    is_zorg=df["beroep_tekst"].str.lower().str.contains("zorg|arts", na=False),
)
```

R

```
microdata |>
  mutate(omzet_per_werknemer = omzet / na_if(personeel, 0),
         grootteklasse = cut(personeel, c(-Inf, 10, 100, Inf), c("klein", "midden", "groot"), right = FALSE),
         sbi_2   = str_sub(sbi, 1, 2),
         is_zorg = str_detect(str_to_lower(beroep_tekst), "zorg|arts"))
```

7. Check for influence and suspiciousness

Local scores per unit per variable = suspiciousness × influence; global score = mean over the variables. Simplified Hidiroglou-Berthelot variant based on the ratio with T-1: - suspiciousness `s = |1 − med/r|` if `r < med`, otherwise `|r/med − 1|`, with `r = y / y_vorig` and `med` the median of `r`; - influence = `gewicht · max(y, y_vorig) / Σ(gewicht · y_vorig)`.

SQL – DuckDB

```
WITH lang AS (
  SELECT id, gewicht, 'omzet' AS variabele, omzet AS y, omzet_vorig AS y_vorig FROM microdata
  UNION ALL
  SELECT id, gewicht, 'personeel', personeel, personeel_vorig FROM microdata
), ratio AS (
  SELECT *, CAST(y AS DOUBLE) / NULLIF(y_vorig, 0) AS r FROM lang
), agg AS (
  SELECT variabele,
         median(CASE WHEN r > 0 THEN r END) AS med_r,
         SUM(gewicht * y_vorig)             AS totaal_vorig
  FROM ratio GROUP BY variabele
), score AS (
  SELECT r.id, r.variabele,
         CASE WHEN r.r IS NULL OR r.r <= 0 THEN NULL
              WHEN r.r < a.med_r THEN ABS(1 - a.med_r / r.r)
              ELSE ABS(r.r / a.med_r - 1) END                        AS verdachtheid,
         r.gewicht * GREATEST(r.y, r.y_vorig) / a.totaal_vorig       AS invloed
  FROM ratio r JOIN agg a ON r.variabele = a.variabele
)
SELECT id, variabele, verdachtheid, invloed,
       verdachtheid * invloed                                  AS lokale_score,
       AVG(verdachtheid * invloed) OVER (PARTITION BY id)      AS globale_score
FROM score
ORDER BY globale_score DESC;
```

SQL – Trino (same query; only the median differs)

```
WITH lang AS (
  SELECT id, gewicht, 'omzet' AS variabele, omzet AS y, omzet_vorig AS y_vorig FROM microdata
  UNION ALL
  SELECT id, gewicht, 'personeel', personeel, personeel_vorig FROM microdata
), ratio AS (
  SELECT *, CAST(y AS DOUBLE) / NULLIF(y_vorig, 0) AS r FROM lang
), agg AS (
  SELECT variabele,
         approx_percentile(CASE WHEN r > 0 THEN r END, 0.5) AS med_r,   -- <- difference from DuckDB
         SUM(gewicht * y_vorig)                             AS totaal_vorig
  FROM ratio GROUP BY variabele
), score AS (
  SELECT r.id, r.variabele,
         CASE WHEN r.r IS NULL OR r.r <= 0 THEN NULL
              WHEN r.r < a.med_r THEN ABS(1 - a.med_r / r.r)
              ELSE ABS(r.r / a.med_r - 1) END                        AS verdachtheid,
         r.gewicht * GREATEST(r.y, r.y_vorig) / a.totaal_vorig       AS invloed
  FROM ratio r JOIN agg a ON r.variabele = a.variabele
)
SELECT id, variabele, verdachtheid, invloed,
       verdachtheid * invloed                                  AS lokale_score,
       AVG(verdachtheid * invloed) OVER (PARTITION BY id)      AS globale_score
FROM score
ORDER BY globale_score DESC;
```

Python

```
def lokale_scores(df, variabelen, gewicht="gewicht"):
    uit = []
    for v in variabelen:
        y, y0 = df[v], df[f"{v}_vorig"]
        r = (y / y0.where(y0 != 0)).where(lambda s: s > 0)
        med = r.median()
        s = pd.Series(np.abs(np.where(r < med, 1 - med / r, r / med - 1)), index=df.index).where(r.notna())
        invloed = df[gewicht] * np.maximum(y, y0) / (df[gewicht] * y0).sum()
        uit.append(pd.DataFrame({"id": df["id"], "variabele": v, "verdachtheid": s,
                                 "invloed": invloed, "lokale_score": s * invloed}))
    out = pd.concat(uit, ignore_index=True)
    out["globale_score"] = out.groupby("id")["lokale_score"].transform("mean")
    return out

scores = lokale_scores(df, ["omzet", "personeel"])
```

R

```
lokale_scores <- function(df, variabelen) {
  map_dfr(variabelen, function(v) {
    y  <- df[[v]];  y0 <- df[[paste0(v, "_vorig")]]
    r  <- y / ifelse(y0 == 0, NA, y0);  r[!is.na(r) & r <= 0] <- NA
    med <- median(r, na.rm = TRUE)
    s  <- abs(ifelse(r < med, 1 - med / r, r / med - 1))
    invloed <- df$gewicht * pmax(y, y0) / sum(df$gewicht * y0, na.rm = TRUE)
    tibble(id = df$id, variabele = v, verdachtheid = s, invloed = invloed, lokale_score = s * invloed)
  }) |>
    group_by(id) |>
    mutate(globale_score = mean(lokale_score, na.rm = TRUE)) |>
    ungroup()
}
scores <- lokale_scores(microdata, c("omzet", "personeel"))
```

---

8. Derive variables (changing measurement scale)

Ratio scale → ordinal categories via thresholds (here 100,000 and 1,000,000).

SQL – DuckDB & Trino

```
SELECT m.*,
       CASE WHEN omzet IS NULL     THEN NULL
            WHEN omzet < 100000    THEN '1: klein'
            WHEN omzet < 1000000   THEN '2: midden'
            ELSE                        '3: groot' END AS omzetklasse
FROM microdata m;
```

Python

```
df["omzetklasse"] = pd.cut(df["omzet"], bins=[-np.inf, 1e5, 1e6, np.inf],
                           labels=["1: klein", "2: midden", "3: groot"], right=False, ordered=True)
```

R

```
microdata |>
  mutate(omzetklasse = cut(omzet, breaks = c(-Inf, 1e5, 1e6, Inf),
                           labels = c("1: klein", "2: midden", "3: groot"),
                           right = FALSE, ordered_result = TRUE))
```

9. Weighting / grossing up

Post-stratification: weight = population count / sample count per stratum. Population table `populatie(sbi, n_pop)`.

SQL – DuckDB & Trino

```
WITH steekproef AS (
  SELECT sbi, COUNT(*) AS n_steekproef FROM microdata GROUP BY sbi
)
SELECT m.*, CAST(p.n_pop AS DOUBLE) / s.n_steekproef AS gewicht_nieuw
FROM microdata m
JOIN steekproef s ON m.sbi = s.sbi
JOIN populatie  p ON m.sbi = p.sbi;
```

Python

```
n_steekproef = df.groupby("sbi")["id"].transform("count")
n_pop = df["sbi"].map(populatie.set_index("sbi")["n_pop"])
df["gewicht_nieuw"] = n_pop / n_steekproef
```

R

```
microdata |>
  add_count(sbi, name = "n_steekproef") |>
  left_join(populatie, by = "sbi") |>
  mutate(gewicht_nieuw = n_pop / n_steekproef)
# multiple auxiliary variables / calibration: survey::postStratify(), survey::calibrate(), sampling::calib()
```

10. Aggregation

Micro → dimensional dataset: classify units by dimensions (`sbi`, `gemeente`) and sum the (weighted) measures.

SQL – DuckDB & Trino

```
SELECT sbi, gemeente,
       COUNT(*)              AS aantal_eenheden,
       SUM(gewicht)          AS aantal_geschat,
       SUM(gewicht * omzet)  AS omzet_gewogen,
       SUM(omzet)            AS omzet_ongewogen
FROM microdata
GROUP BY sbi, gemeente;
```

Python

```
agg = (df.assign(omzet_gew=df["omzet"] * df["gewicht"])
         .groupby(["sbi", "gemeente"], as_index=False)
         .agg(aantal_eenheden=("id", "count"), aantal_geschat=("gewicht", "sum"),
              omzet_gewogen=("omzet_gew", "sum"), omzet_ongewogen=("omzet", "sum")))
```

R

```
agg <- microdata |>
  group_by(sbi, gemeente) |>
  summarise(aantal_eenheden = n(), aantal_geschat = sum(gewicht),
            omzet_gewogen = sum(gewicht * omzet, na.rm = TRUE),
            omzet_ongewogen = sum(omzet, na.rm = TRUE), .groups = "drop")
```

> Quality metadata: aggregate an indicator, e.g. `AVG(CASE WHEN omzet_valide THEN 1.0 ELSE 0.0 END)` (SQL), `.mean()` (pandas) or `mean(omzet_valide)` (R).

---

11. Fitting (Stone)

Adjust values so that linear constraints $A \cdot x = b$ hold: $x' = x + V A^T (A V A^T)^{-1} (b - A x)$, with $V = \mathrm{diag}(x)$ (variance proportional to size). Example: `loon + inkoop + overig = totaal` (wages + purchases + other = total) with table `kosten_detail(id, loon, inkoop, overig, totaal)`.

SQL – DuckDB & Trino (a single sum constraint only; in that case the Stone solution with $V = \mathrm{diag}(x)$ is a proportional scaling. Multiple constraints → Python/R)

```
SELECT id,
       loon   * totaal / (loon + inkoop + overig) AS loon_ingepast,
       inkoop * totaal / (loon + inkoop + overig) AS inkoop_ingepast,
       overig * totaal / (loon + inkoop + overig) AS overig_ingepast,
       totaal
FROM kosten_detail
WHERE loon + inkoop + overig <> 0;
```

Note: columns must be `DOUBLE`/`DECIMAL` (with integers, Trino uses integer division).

Python (general, any number of constraints)

```
def stone(x, A, b, V=None):
    x = np.asarray(x, float)
    V = np.diag(x) if V is None else V
    return x + V @ A.T @ np.linalg.solve(A @ V @ A.T, np.asarray(b, float) - A @ x)

A = np.array([[1.0, 1.0, 1.0]])                                  # loon + inkoop + overig
rij = df[["loon", "inkoop", "overig", "totaal"]].to_numpy(float)
df[["loon_ingepast", "inkoop_ingepast", "overig_ingepast"]] = [stone(r[:3], A, [r[3]]) for r in rij]
```

R

```
stone <- function(x, A, b, V = diag(x, nrow = length(x))) {
  x + V %*% t(A) %*% solve(A %*% V %*% t(A), b - A %*% x)
}
A <- matrix(1, nrow = 1, ncol = 3)
ingepast <- t(apply(kosten_detail[, c("loon", "inkoop", "overig", "totaal")], 1,
                    function(r) as.numeric(stone(r[1:3], A, r[4]))))
kosten_detail[, c("loon_ingepast", "inkoop_ingepast", "overig_ingepast")] <- ingepast
```

---

12. Check plausibility

Value against a computed range: here the IQR bounds (Q1 − 1.5·IQR, Q3 + 1.5·IQR) per SBI.

SQL – DuckDB

```
WITH grenzen AS (
  SELECT sbi,
         quantile_cont(omzet, 0.25) AS q1,
         quantile_cont(omzet, 0.75) AS q3
  FROM microdata GROUP BY sbi
)
SELECT m.*,
       COALESCE(m.omzet BETWEEN g.q1 - 1.5 * (g.q3 - g.q1)
                            AND g.q3 + 1.5 * (g.q3 - g.q1), FALSE) AS omzet_plausibel
FROM microdata m
LEFT JOIN grenzen g ON m.sbi = g.sbi;
```

SQL – Trino

```
WITH grenzen AS (
  SELECT sbi,
         approx_percentile(omzet, 0.25) AS q1,
         approx_percentile(omzet, 0.75) AS q3
  FROM microdata GROUP BY sbi
)
SELECT m.*,
       COALESCE(m.omzet BETWEEN g.q1 - 1.5 * (g.q3 - g.q1)
                            AND g.q3 + 1.5 * (g.q3 - g.q1), FALSE) AS omzet_plausibel
FROM microdata m
LEFT JOIN grenzen g ON m.sbi = g.sbi;
```

Python

```
q1 = df.groupby("sbi")["omzet"].transform(lambda s: s.quantile(0.25))
q3 = df.groupby("sbi")["omzet"].transform(lambda s: s.quantile(0.75))
iqr = q3 - q1
df["omzet_plausibel"] = df["omzet"].between(q1 - 1.5 * iqr, q3 + 1.5 * iqr)
```

R

```
microdata |>
  group_by(sbi) |>
  mutate(q1 = quantile(omzet, 0.25, na.rm = TRUE), q3 = quantile(omzet, 0.75, na.rm = TRUE),
         omzet_plausibel = !is.na(omzet) & between(omzet, q1 - 1.5 * (q3 - q1), q3 + 1.5 * (q3 - q1))) |>
  ungroup() |>
  select(-q1, -q3)
```

---

13. Derive variables and units

New object type: from persons to households. Table `personen(persoon_id, huishouden_id, leeftijd, inkomen)` → one row per household.

SQL – DuckDB & Trino

```sql
SELECT huishouden_id,
       COUNT(*)                                        AS aantal_personen,
       AVG(leeftijd)                                   AS gem_leeftijd,
       MAX(leeftijd)                                   AS leeftijd_oudste,
       SUM(CASE WHEN leeftijd < 18 THEN 1 ELSE 0 END)  AS aantal_kinderen,
       SUM(inkomen)                                    AS huishoudinkomen
FROM personen
GROUP BY huishouden_id;
```

Python

```python
huishoudens = (personen.assign(kind=personen["leeftijd"] < 18)
               .groupby("huishouden_id", as_index=False)
               .agg(aantal_personen=("persoon_id", "count"), gem_leeftijd=("leeftijd", "mean"),
                    leeftijd_oudste=("leeftijd", "max"), aantal_kinderen=("kind", "sum"),
                    huishoudinkomen=("inkomen", "sum")))
```

R

```r
huishoudens <- personen |>
  group_by(huishouden_id) |>
  summarise(aantal_personen = n(), gem_leeftijd = mean(leeftijd), leeftijd_oudste = max(leeftijd),
            aantal_kinderen = sum(leeftijd < 18), huishoudinkomen = sum(inkomen), .groups = "drop")
```

---

14. Derive variables

New property with its own value domain, same object type (here profit margin and turnover growth).

SQL – DuckDB & Trino

```sql
SELECT m.*,
       CAST(winst AS DOUBLE) / NULLIF(omzet, 0)                     AS winstmarge,
       CAST(omzet - omzet_vorig AS DOUBLE) / NULLIF(omzet_vorig, 0)  AS omzetgroei
FROM microdata m;
```

Python

```python
df["winstmarge"] = df["winst"] / df["omzet"].where(df["omzet"] != 0)
df["omzetgroei"] = (df["omzet"] - df["omzet_vorig"]) / df["omzet_vorig"].where(df["omzet_vorig"] != 0)
```

R

```r
microdata |>
  mutate(winstmarge = winst / na_if(omzet, 0),
         omzetgroei = (omzet - omzet_vorig) / na_if(omzet_vorig, 0))
```

---

15. Link on foreign key

`bedrijven(bedrijf_id, naam, gemeente_code)` (businesses) has a foreign key to `gemeenten(gemeente_code, gemeente_naam, provincie)` (municipalities). Dataset 1 stays leading; rows from dataset 2 are repeated for duplicate key values.

SQL – DuckDB & Trino

```
SELECT b.*, g.gemeente_naam, g.provincie
FROM bedrijven b
LEFT JOIN gemeenten g ON b.gemeente_code = g.gemeente_code;
```

Python

```
resultaat = bedrijven.merge(gemeenten, on="gemeente_code", how="left", validate="m:1")
```

R

```
resultaat <- bedrijven |>
  left_join(gemeenten, by = "gemeente_code", relationship = "many-to-one")
```

---

16. Link on primary key

`dataset1(id, omzet)` and `dataset2(id, personeel)`, both unique on `id`. Choose the join type: left, right, inner or full outer.

SQL – DuckDB & Trino

```
SELECT COALESCE(d1.id, d2.id) AS id, d1.omzet, d2.personeel
FROM dataset1 d1
LEFT JOIN dataset2 d2 ON d1.id = d2.id;      -- RIGHT JOIN / INNER JOIN / FULL OUTER JOIN
```

Python

```
resultaat = d1.merge(d2, on="id", how="left", validate="1:1")   # "right" | "inner" | "outer"
```

R

```
resultaat <- left_join(d1, d2, by = "id", relationship = "one-to-one")
# right_join() | inner_join() | full_join()
```

---

17. Select variables

SQL – DuckDB

```
SELECT id, sbi, omzet FROM microdata;

-- DuckDB extras
SELECT * EXCLUDE (beroep_tekst, gewicht) FROM microdata;   -- everything except ...
SELECT id, COLUMNS('^omzet') FROM microdata;               -- by pattern
```

SQL – Trino

```
SELECT id, sbi, omzet FROM microdata;
```

Python

```
selectie = df[["id", "sbi", "omzet"]]
# everything except: df.drop(columns=["beroep_tekst", "gewicht"])
```

R

```
selectie <- microdata |> select(id, sbi, omzet)
# everything except: select(-beroep_tekst, -gewicht)
```

---

18. Recode

Replace the codes of a variable via a key `koppelsleutel(code_oud, code_nieuw)`. "Verrinnen" = BSN (citizen service number) → CBS-wide RIN (random identifying number); "verprinnen" = CBS RIN → project-specific RIN. Example: `persoon_data(bsn, leeftijd, inkomen)` and key `bsn_rin(bsn, rin)`.

SQL – DuckDB

```
-- recode/verrinnen: replace the identifying column, other columns remain
SELECT k.rin, p.* EXCLUDE (bsn)
FROM persoon_data p
LEFT JOIN bsn_rin k ON p.bsn = k.bsn;

-- verprinnen: create project-specific RINs (store the key separately)
CREATE TABLE rin_project AS
SELECT rin, ROW_NUMBER() OVER (ORDER BY random()) AS project_rin
FROM (SELECT DISTINCT rin FROM bsn_rin);
```

SQL – Trino

```
SELECT k.rin, p.leeftijd, p.inkomen            -- columns listed explicitly (no EXCLUDE in Trino)
FROM persoon_data p
LEFT JOIN bsn_rin k ON p.bsn = k.bsn;

CREATE TABLE rin_project AS
SELECT rin, ROW_NUMBER() OVER (ORDER BY random()) AS project_rin
FROM (SELECT DISTINCT rin FROM bsn_rin);
```

Python

```
omgecodeerd = (persoon_data.merge(bsn_rin, on="bsn", how="left")
                           .drop(columns="bsn"))

rng = np.random.default_rng()                                  # verprinnen
rin_project = pd.DataFrame({"rin": bsn_rin["rin"].unique()})
rin_project["project_rin"] = rng.permutation(len(rin_project)) + 1
```

R

```
omgecodeerd <- persoon_data |>
  left_join(bsn_rin, by = "bsn") |>
  select(-bsn)

rin_project <- tibble(rin = unique(bsn_rin$rin)) |>            # verprinnen
  mutate(project_rin = sample(n()))
```

---

19. Select units

Select rows based on value-range restrictions; the columns remain the same.

SQL – DuckDB & Trino

```
SELECT *
FROM microdata
WHERE sbi LIKE '47%'
  AND omzet >= 100000
  AND gemeente IN ('0363', '0599');
```

Python

```
selectie = df[df["sbi"].str.startswith("47") & (df["omzet"] >= 100_000) & df["gemeente"].isin(["0363", "0599"])]
```

R

```
selectie <- microdata |>
  filter(str_starts(sbi, "47"), omzet >= 100000, gemeente %in% c("0363", "0599"))
```

---

20. Combine (complement)

Stack the rows of two datasets with identical columns.

SQL – DuckDB

```
SELECT * FROM dataset_a
UNION ALL BY NAME
SELECT * FROM dataset_b;
-- without duplicate rows: UNION BY NAME
```

SQL – Trino

```
SELECT id, omzet, personeel FROM dataset_a     -- column order and types must match
UNION ALL
SELECT id, omzet, personeel FROM dataset_b;
-- without duplicate rows: UNION
```

Python

```
samen = pd.concat([dataset_a, dataset_b], ignore_index=True)
# without duplicates: .drop_duplicates()
```

R

```
samen <- bind_rows(dataset_a, dataset_b)
# without duplicates: distinct()
```